INDEX

S.No	List of Experiments	Page No
1	Implementation of a symbol table	2
2	Lexical analyzer to recognize identifiers, constants, comments, operators, etc.	4
3	Implementation of a lexical analyzer using the Lex tool	7
4	Generate YACC specification for a few syntactic categories	9
5	Convert BNF rules into YACC form and generate an abstract syntax tree	13
6	Implement type checking	16
7	Implement a storage allocation strategy (heap)	18
8	Lex program to count the number of words and lines	20
9	Lexical analyzer using a C program	22
10	Recursive descent parser for E → E+T | T, T → T*F | F, F → (E) | id	25
11	Recursive descent parser for S → (L) | a, L → L,S | S	28
12	C program to calculate the FIRST function of a grammar	31
13	YACC program to implement a top-down parser	33
14	YACC program to evaluate an algebraic expression	35

EXPERIMENT 1

Implementation of a symbol table

AIM

Implementation of a symbol table.

PROGRAM

1.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct {
    char symbol[20];
    char *address;
    char type[20];
} Symbol;

int main() {
    char exp[100], c;
    Symbol table[20];
    int i = 0, x = 0;

    printf("Enter expression (end with $): ");
    scanf("%99s", exp);

    printf("\nGiven Expression: %s\n", exp);
    printf("\nSymbol\tAddress\t\tType\n");

    for (i = 0; exp[i] != '\0' && exp[i] != '$' && x < 20; i++) {
        c = exp[i];

        if (isalpha((unsigned char)c)) {
            table[x].symbol[0] = c;
            table[x].symbol[1] = '\0';
            table[x].address = &exp[i];
            strcpy(table[x].type, "Identifier");

            printf("%c\t%p\t%s\n",
                   table[x].symbol[0],
                   (void *)table[x].address,
                   table[x].type);
            x++;
        }
        else if (c == '+' || c == '-' || c == '*' ||
                 c == '/' || c == '=') {
            table[x].symbol[0] = c;
            table[x].symbol[1] = '\0';
            table[x].address = &exp[i];
            strcpy(table[x].type, "Operator");

            printf("%c\t%p\t%s\n",
                   table[x].symbol[0],
                   (void *)table[x].address,
                   table[x].type);
            x++;
        }
    }
```

```
    return 0;
}
```

COMPILE & RUN

```
> gcc 1.c -o 1.exe
> 1.exe
Enter expression (end with $): a=b+c$

Given Expression: a=b+c$

Symbol  Address         Type
a       0x7ffd406ff800  Identifier
=       0x7ffd406ff801  Operator
b       0x7ffd406ff802  Identifier
+       0x7ffd406ff803  Operator
c       0x7ffd406ff804  Identifier
```

VIVA QUESTIONS

Q1. What is a symbol table?

A: A data structure used by the compiler to store information about identifiers such as variables, functions, constants and their attributes.

Q2. Why is a symbol table required?

A: For fast lookup: to check declaration before use, determine scope, perform type checking and allocate memory during compilation.

Q3. What are the main operations on a symbol table?

A: Insert(symbol, attributes) and Lookup(symbol).

EXPERIMENT 2

Lexical analyzer to recognize identifiers, constants, comments, operators, etc.

AIM

Develop a lexical analyzer to recognize a few patterns, e.g. identifiers, constants, comments, operators, etc.

PROGRAM

2.c

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char *word) {
    char keywords[][10] = {
        "int", "char", "float", "double", "void",
        "if", "else", "for", "while", "return",
        "break", "continue", "do", "long", "short"
    };

    int i;
    for (i = 0; i < 15; i++) {
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

int main() {
    char line[1000], buf[1000];
    int i = 0, p = 0;

    printf("Enter a line of code:\n");
    fgets(line, sizeof(line), stdin);

    printf("\n--- Lexical Analysis Output ---\n");

    for (p = 0; line[p] != '\0'; p++) {
        char ch = line[p];

        if (isalpha((unsigned char)ch) || ch == '_') {
            i = 0;
            buf[i++] = ch;

            while (isalnum((unsigned char)line[p + 1]) ||
                   line[p + 1] == '_') {
                buf[i++] = line[++p];
            }

            buf[i] = '\0';

            if (isKeyword(buf))
                printf("[Keyword : %s]\n", buf);
            else
                printf("[Identifier : %s]\n", buf);
        }
        else if (isdigit((unsigned char)ch)) {
            i = 0;
            buf[i++] = ch;
```

```
            while (isdigit((unsigned char)line[p + 1]) ||
                   line[p + 1] == '.') {
                buf[i++] = line[++p];
            }

            buf[i] = '\0';
            printf("[Constant : %s]\n", buf);
        }
        else if (ch == '/' && line[p + 1] == '/') {
            printf("[Single-line Comment]\n");
            break;
        }
        else if (strchr("=+-*/<>!", ch) != NULL) {
            if (line[p + 1] == '=')
                printf("[Operator : %c%c]\n", ch, line[++p]);
            else
                printf("[Operator : %c]\n", ch);
        }
        else if (strchr(";,(){}[]", ch) != NULL) {
            printf("[Delimiter : %c]\n", ch);
        }
        else if (isspace((unsigned char)ch)) {
            continue;
        }
        else {
            printf("[Unknown : %c]\n", ch);
        }
    }

    return 0;
}
```

COMPILE & RUN

```
> gcc 2.c -o 2.exe
> 2.exe
Enter a line of code:
int a = 10;

--- Lexical Analysis Output ---
[Keyword : int]
[Identifier : a]
[Operator : =]
[Constant : 10]
[Delimiter : ;]

> 2.exe
Enter a line of code:
float x1 = a >= 2.5; // test

--- Lexical Analysis Output ---
[Keyword : float]
[Identifier : x1]
[Operator : =]
[Identifier : a]
[Operator : >=]
[Constant : 2.5]
[Delimiter : ;]
[Single-line Comment]
```

VIVA QUESTIONS

Q1. What is a lexical analyzer?

A: The first phase of a compiler, also called a scanner. It reads the source code character by character and converts it into a stream of tokens.

Q2. What is the main role of a lexical analyzer?

A: Identify tokens (keywords, identifiers, operators, constants); remove whitespace, tabs and newlines; strip comments; interact with the symbol table to store identifiers.

Q3. Define token, lexeme and pattern.

A: Token: a category for a logical unit (KEYWORD, IDENTIFIER, OPERATOR). Lexeme: the actual character sequence in the source that matches a pattern (x, int, 100). Pattern: the rule describing the form of lexemes, usually written as a regular expression.

Q4. What is the output of a lexical analyzer?

A: A stream of tokens, which is passed to the syntax analyzer (parser).

EXPERIMENT 3

Implementation of a lexical analyzer using the Lex tool

AIM

Implementation of a lexical analyzer using the Lex tool.

PROGRAM

lexer.l

```
%{
#include <stdio.h>
%}

DIGIT   [0-9]
ID      [a-zA-Z_][a-zA-Z0-9_]*
KEYWORD (if|else|while|for|return|int|char|float|double|void)
OPERATOR [+\-*/=<>&|!]+
SEPARATOR [(){};,]

%%
{KEYWORD}       { printf("Keyword: %s\n", yytext); }
{ID}            { printf("Identifier: %s\n", yytext); }
{DIGIT}+        { printf("Number: %s\n", yytext); }
{OPERATOR}      { printf("Operator: %s\n", yytext); }
{SEPARATOR}     { printf("Separator: %s\n", yytext); }
[ \t\n]+        ;
.               { printf("Unknown: %s\n", yytext); }
%%

int main()
{
    printf("Enter code:\n");
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}
```

COMPILE & RUN

```
> flex lexer.l
> gcc lex.yy.c -o lexer.exe
> lexer.exe
Enter code:
int a = 10 + 20;
Keyword: int
Identifier: a
Operator: =
Number: 10
Operator: +
Number: 20
Separator: ;
^Z
```

VIVA QUESTIONS

Q1. What is a lexical analyzer (scanner)?

A: The first phase of a compiler. It groups characters of the source program into lexemes, converts them into tokens and removes comments and white space.

Q2. What are the sections of a Lex program?

A: Definitions section (headers, macros), rules section (pattern-action pairs) and user-code section (main(), yywrap()), separated by %%.

Q3. What is yylex()?

A: The scanner function generated by Lex. It reads the input, matches it against the rules and returns a token whenever a match is found.

Q4. What is the use of yywrap()?

A: It is called at end of input. Returning 1 tells the scanner there is no more input to process.

EXPERIMENT 4

Generate YACC specification for a few syntactic categories

AIM

Generate YACC specification for a few syntactic categories.

4a) Program to recognize a valid arithmetic expression that uses the operators +, -, * and /.

PROGRAM

exp4a.l

```
%{
#include "y.tab.h"
%}

%%
[0-9]+      { return NUMBER; }
[+\-*/]     { return yytext[0]; }
[(]         { return '('; }
[)]         { return ')'; }
[ \t\n]     ;
.           { return yytext[0]; }
%%

int yywrap()
{
    return 1;
}
```

exp4a.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
int yyerror(char *s);
%}

%token NUMBER
%left '+' '-'
%left '*' '/'

%%
expr:
      expr '+' expr
    | expr '-' expr
    | expr '*' expr
    | expr '/' expr
    | '(' expr ')'
    | NUMBER
    ;
%%

int main()
{
    printf("Enter an arithmetic expression:\n");
    if (yyparse() == 0)
        printf("Valid arithmetic expression\n");
    return 0;
}
```

```
int yyerror(char *s)
{
    printf("Invalid arithmetic expression\n");
    return 0;
}
```

COMPILE & RUN

```
> bison -dy exp4a.y
> flex exp4a.l
> gcc y.tab.c lex.yy.c -o exp4a.exe
> exp4a.exe
Enter an arithmetic expression:
3 + 4 * (2 - 1)
^Z
Valid arithmetic expression

> exp4a.exe
Enter an arithmetic expression:
3 + * 4
Invalid arithmetic expression
```

4b) Program to recognize a valid variable which starts with a letter followed by any number of letters or digits.

PROGRAM

4b.c

```c
#include <stdio.h>
#include <ctype.h>

int main()
{
    char var[100];
    int i = 0, valid = 1;

    printf("Enter a variable name: ");
    scanf("%99s", var);

    if (!isalpha((unsigned char)var[0]))
    {
        valid = 0;
    }
    else
    {
        while (var[i] != '\0')
        {
            if (!isalnum((unsigned char)var[i]))
            {
                valid = 0;
                break;
            }
            i++;
        }
    }

    if (valid)
        printf("Valid variable name\n");
    else
        printf("Invalid variable name\n");
```

```
    return 0;
}
```

COMPILE & RUN

```
> gcc 4b.c -o 4b.exe
> 4b.exe
Enter a variable name: total1
Valid variable name

> 4b.exe
Enter a variable name: total_1
Invalid variable name

> 4b.exe
Enter a variable name: 9abc
Invalid variable name
```

4c) Implementation of a calculator using Lex and YACC.

PROGRAM

exp4c.l

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t]       ;
\n          { return '\n'; }
.           { return yytext[0]; }
%%

int yywrap()
{
    return 1;
}
```

exp4c.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
%}

%token NUMBER
%left '+' '-'
%left '*' '/'
%left UMINUS

%%
input:
      /* empty */
    | input expr '\n' { printf("Result = %d\n", $2); }
    ;

expr:
      expr '+' expr  { $$ = $1 + $3; }
```

```
    | expr '-' expr  { $$ = $1 - $3; }
    | expr '*' expr  { $$ = $1 * $3; }
    | expr '/' expr  {
          if ($3 == 0) {
              yyerror("division by zero");
              $$ = 0;
          } else {
              $$ = $1 / $3;
          }
      }
    | '-' expr %prec UMINUS { $$ = -$2; }
    | '(' expr ')' { $$ = $2; }
    | NUMBER       { $$ = $1; }
    ;
%%

void yyerror(const char *s)
{
    printf("Error: %s\n", s);
}

int main()
{
    printf("Enter expressions (one per line):\n");
    yyparse();
    return 0;
}
```

COMPILE & RUN

```
> bison -dy exp4c.y
> flex exp4c.l
> gcc y.tab.c lex.yy.c -o exp4c.exe
> exp4c.exe
Enter expressions (one per line):
5+3
Result = 8
10-2*3
Result = 4
(4+6)/2
Result = 5
-5+2
Result = -3
^Z
```

VIVA QUESTIONS

Q1. What is a variable in C?

A: A named memory location used to store data values.

Q2. What is YACC?

A: Yet Another Compiler Compiler: a parser generator that checks a grammar and produces an LALR parser.

Q3. What is a token?

A: A meaningful sequence of characters identified by Lex (e.g. NUMBER, +, -).

Q4. What do %left '+' '-' and %left '*' '/' do?

A: They declare operator associativity and precedence. Operators declared later (*, /) have higher precedence than earlier ones (+, -).

EXPERIMENT 5

Convert BNF rules into YACC form and generate an abstract syntax tree

AIM

Convert the BNF rules into YACC form and write code to generate an abstract syntax tree.

```
expr ::= expr + term | expr - term | term
term ::= term * factor | term / factor | factor
factor ::= ( expr ) | number
```

PROGRAM

exp5.l

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+    { yylval.num = atoi(yytext); return NUMBER; }
[ \t]     ;
\n        { return '\n'; }
.         { return yytext[0]; }
%%

int yywrap()
{
    return 1;
}
```

exp5.y

```
%code requires {
typedef struct ast {
    char nodetype;
    struct ast *left;
    struct ast *right;
    int value;
} AST;
}

%{
#include <stdio.h>
#include <stdlib.h>
%}

%union {
    int num;
    AST *node;
}

%code {
AST *new_node(char nodetype, AST *left,
              AST *right, int value);
void print_ast(AST *node);
int yylex(void);
void yyerror(const char *msg);
}

%token <num> NUMBER
```

```
%type <node> expr term factor
%left '+' '-'
%left '*' '/'

%%
input:
    expr '\n' {
        printf("Parsed AST: ");
        print_ast($1);
        printf("\n");
    }
    ;

expr:
      expr '+' term {
          $$ = new_node('+', $1, $3, 0);
      }
    | expr '-' term {
          $$ = new_node('-', $1, $3, 0);
      }
    | term {
          $$ = $1;
      }
    ;

term:
      term '*' factor {
          $$ = new_node('*', $1, $3, 0);
      }
    | term '/' factor {
          $$ = new_node('/', $1, $3, 0);
      }
    | factor {
          $$ = $1;
      }
    ;

factor:
      '(' expr ')' {
          $$ = $2;
      }
    | NUMBER {
          $$ = new_node('N', NULL, NULL, $1);
      }
    ;
%%

AST *new_node(char nodetype, AST *left,
              AST *right, int value)
{
    AST *node = malloc(sizeof(AST));

    if (node == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }

    node->nodetype = nodetype;
    node->left = left;
    node->right = right;
    node->value = value;

    return node;
}

void print_ast(AST *node)
```

```
{
    if (node == NULL)
        return;

    if (node->nodetype == 'N') {
        printf("%d", node->value);
    } else {
        printf("(");
        print_ast(node->left);
        printf(" %c ", node->nodetype);
        print_ast(node->right);
        printf(")");
    }
}

void yyerror(const char *msg)
{
    printf("Syntax error: %s\n", msg);
}

int main()
{
    printf("Enter expression:\n");
    yyparse();
    return 0;
}
```

COMPILE & RUN

```
> bison -dy exp5.y
> flex exp5.l
> gcc y.tab.c lex.yy.c -o exp5.exe
> exp5.exe
Enter expression:
3+4*(2-1)
Parsed AST: (3 + (4 * (2 - 1)))
^Z

> exp5.exe
Enter expression:
2*(3+4)-5
Parsed AST: ((2 * (3 + 4)) - 5)
^Z
```

VIVA QUESTIONS

Q1. What is BNF?

A: Backus–Naur Form: a notation used to describe context-free grammar rules.

Q2. What is YACC?

A: Yet Another Compiler Compiler: a parser generator that produces LALR parsers.

Q3. What is an abstract syntax tree?

A: A tree representation of the syntactic structure of the source where each node is an operator and its children are operands; unnecessary grammar details (parentheses, chain rules) are left out.

EXPERIMENT 6

Implement type checking

AIM

Implement type checking.

PROGRAM

exp6.c

```c
#include <stdio.h>

typedef enum {
    TYPE_INT,
    TYPE_FLOAT,
    TYPE_CHAR
} Type;

typedef struct {
    Type type;
    void *value;
} TypedVar;

int add(int a, int b)
{
    return a + b;
}

float multiply(float a, float b)
{
    return a * b;
}

float divide(int a, int b)
{
    if (b != 0)
        return (float)a / b;
    else
        return 0.0f;
}

float average(int arr[], int n)
{
    float sum = 0;
    int i;

    for (i = 0; i < n; i++)
        sum += arr[i];

    return sum / n;
}

void print_var(TypedVar var)
{
    switch (var.type) {
        case TYPE_INT:
            printf("Integer: %d\n", *(int *)var.value);
            break;

        case TYPE_FLOAT:
            printf("Float: %.2f\n", *(float *)var.value);
            break;
```

```
        case TYPE_CHAR:
            printf("Character: %c\n", *(char *)var.value);
            break;
    }
}

int main()
{
    int a = 20, x = 5, y = 10;
    int arr[5] = {1, 2, 3, 4, 5};

    float b = 2.718f, m = 2.5f, n = 4.0f;
    char c = 'Z';

    TypedVar va = {TYPE_INT, &a};
    TypedVar vb = {TYPE_FLOAT, &b};
    TypedVar vc = {TYPE_CHAR, &c};

    print_var(va);
    print_var(vb);
    print_var(vc);

    printf("Sum of %d and %d = %d\n", x, y, add(x, y));
    printf("Product of %.2f and %.2f = %.2f\n",
           m, n, multiply(m, n));
    printf("Division result = %.2f\n", divide(a, 2));
    printf("Average of integer array = %.2f\n",
           average(arr, 5));

    return 0;
}
```

COMPILE & RUN

```
> gcc exp6.c -o exp6.exe
> exp6.exe
Integer: 20
Float: 2.72
Character: Z
Sum of 5 and 10 = 15
Product of 2.50 and 4.00 = 10.00
Division result = 10.00
Average of integer array = 3.00
```

VIVA QUESTIONS

Q1. What is type checking?

A: A semantic-analysis process that ensures the program follows the language's type rules, such as operand compatibility.

Q2. When does it occur?

A: During compilation (static) or at run time (dynamic).

Q3. Static vs. dynamic type checking?

A: Static: done at compile time (C, Java). Dynamic: done at run time (Python, JavaScript).

EXPERIMENT 7

Implement a storage allocation strategy (heap)

AIM

Implement any one storage allocation strategy (heap, stack or static). This program uses heap (dynamic) allocation with malloc() and free().

PROGRAM

exp7.c

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *arr;
    int n;
    int i;

    printf("Enter no of elements: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Invalid number of elements\n");
        return 1;
    }

    arr = (int *)malloc(n * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    int fake_address = 100;

    printf("\nElement\tValue\tSimulated Address\n");

    for (i = 0; i < n; i++) {
        arr[i] = (i + 1) * 10;

        printf("arr[%d]\t%d\t%d\n",
               i, arr[i], fake_address);

        fake_address += sizeof(int);
    }

    free(arr);

    return 0;
}
```

COMPILE & RUN

```
> gcc exp7.c -o exp7.exe
> exp7.exe
Enter no of elements: 5

Element Value   Simulated Address
arr[0]  10      100
arr[1]  20      104
arr[2]  30      108
arr[3]  40      112
arr[4]  50      116
```

VIVA QUESTIONS

Q1. What is the primary difference between stack and heap allocation?

A: Stack allocation is automatic, LIFO and limited in size, managed by the compiler. Heap allocation is manual (or GC-managed), dynamic, larger and follows no specific order.

Q2. What are the advantages of stack allocation?

A: Very fast access, automatic management and efficient reuse of memory.

Q3. What is garbage collection?

A: Automatic memory management of the heap: the system finds and reclaims memory that the program no longer uses.

EXPERIMENT 8

Lex program to count the number of words and lines

AIM

Write a Lex program to count the number of words and number of lines in a given file or program.

PROGRAM

exp8.l

```
%{
#include <stdio.h>

int word_count = 0;
int line_count = 0;
%}

%%
\n                  { line_count++; }
[A-Za-z0-9]+       { word_count++; }
[ \t]+             ;
.                  ;
%%

int main(void)
{
    printf("Enter text (Ctrl+Z to end input on Windows):\n");

    yylex();

    printf("\nTotal lines: %d\n", line_count);
    printf("Total words: %d\n", word_count);

    return 0;
}

int yywrap(void)
{
    return 1;
}
```

COMPILE & RUN

```
> flex exp8.l
> gcc lex.yy.c -o exp8.exe
> exp8.exe
Enter text (Ctrl+Z to end input on Windows):
hello world
lex is fun
^Z

Total lines: 2
Total words: 5
```

VIVA QUESTIONS

Q1. What is Lex (or Flex)?

A: A scanner generator. It takes regular expressions with actions and generates C code for yylex(), which identifies patterns (tokens) in the input.

Q2. Explain the three sections of a Lex program.

A: Definitions (headers, variables, macros); rules (pattern-action pairs); user code (main(), yywrap() and helper functions). The sections are separated by %%.

Q3. What does yylex() do?

A: It is the main function generated by Lex. It reads characters from the input, matches them against the rules and executes the associated action.

EXPERIMENT 9

Lexical analyzer using a C program

AIM

Write a C program to implement a lexical analyzer. The program reads the source from the file input.txt.

PROGRAM

9.c

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int iskeyword(char buffer[])
{
    char keywords[10][10] = {
        "int", "float", "if", "else", "while",
        "for", "return", "char", "double", "void"
    };

    int i;
    for (i = 0; i < 10; i++)
    {
        if (strcmp(keywords[i], buffer) == 0)
            return 1;
    }
    return 0;
}

int main()
{
    int ch;
    char buffer[20];
    FILE *fp;
    int i = 0;

    fp = fopen("input.txt", "r");

    if (fp == NULL)
    {
        printf("Error opening file\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        if (isalpha(ch) || ch == '_')
        {
            buffer[i++] = ch;

            while ((ch = fgetc(fp)) != EOF &&
                   (isalnum(ch) || ch == '_'))
            {
                if (i < 19)
                    buffer[i++] = ch;
            }

            buffer[i] = '\0';
            i = 0;

            if (iskeyword(buffer))
                printf("%s is keyword\n", buffer);
```

```
            else
                printf("%s is Identifier\n", buffer);

            if (ch == EOF)
                break;
        }

        if (isdigit(ch))
        {
            buffer[i++] = ch;

            while ((ch = fgetc(fp)) != EOF && isdigit(ch))
            {
                if (i < 19)
                    buffer[i++] = ch;
            }

            buffer[i] = '\0';
            i = 0;

            printf("%s is constant\n", buffer);

            if (ch == EOF)
                break;
        }

        if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
            printf("%c is operator\n", ch);
        else if (ch == '=' || ch == ';' || ch == ',' ||
                 ch == '(' || ch == ')')
            printf("%c is special symbol\n", ch);
    }

    fclose(fp);
    return 0;
}
```

input.txt

```
int a = b + 5;
```

COMPILE & RUN

```
> gcc 9.c -o 9.exe
> type input.txt
int a = b + 5;
> 9.exe
int is keyword
a is Identifier
= is special symbol
b is Identifier
+ is operator
5 is constant
; is special symbol
```

VIVA QUESTIONS

Q1. What is a lexical analyzer?

A: The first phase of a compiler; it converts source code into tokens.

Q2. What is a token?

A: The smallest meaningful unit of a program, such as a keyword, identifier, operator or constant.

Q3. How does the program separate keywords from identifiers?

A: It collects a word into a buffer and compares it with a table of reserved keywords using strcmp(); a match is a keyword, otherwise an identifier.

EXPERIMENT 10

Recursive descent parser for E → E+T | T, T → T*F | F, F → (E) | id

AIM

Write a recursive descent parser for the grammar given below.

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

Note: The grammar is left-recursive, so the left recursion is removed (E → T E', E' → + T E' | ε, and similarly for T). In the program E' and T' are implemented as while loops inside E() and T().

PROGRAM

exp10.c

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

enum {
    ID, PLUS, MUL, LPAREN, RPAREN, END, INVALID
};

int token;
char *p;

void next()
{
    while (isspace((unsigned char)*p))
        p++;

    if (isalpha((unsigned char)*p))
    {
        token = ID;
        while (isalnum((unsigned char)*p))
            p++;
    }
    else if (*p == '+')
        token = PLUS, p++;
    else if (*p == '*')
        token = MUL, p++;
    else if (*p == '(')
        token = LPAREN, p++;
    else if (*p == ')')
        token = RPAREN, p++;
    else if (*p == '\0' || *p == '\n')
        token = END;
    else
        token = INVALID, p++;
}

int E();
int T();
int F();

int E()
{
    if (!T())
```

```
        return 0;

    while (token == PLUS)
    {
        next();
        if (!T())
            return 0;
    }
    return 1;
}

int T()
{
    if (!F())
        return 0;

    while (token == MUL)
    {
        next();
        if (!F())
            return 0;
    }
    return 1;
}

int F()
{
    if (token == ID)
    {
        next();
        return 1;
    }

    if (token == LPAREN)
    {
        next();
        if (!E() || token != RPAREN)
            return 0;

        next();
        return 1;
    }

    return 0;
}

int main()
{
    char line[256];

    printf("Enter expression: ");
    if (!fgets(line, sizeof(line), stdin))
        return 1;

    p = line;
    next();

    if (E() && token == END)
        printf("Parsing succeeded: valid expression\n");
    else
        printf("Parsing failed: invalid expression\n");

    return 0;
}
```

COMPILE & RUN

```
> gcc exp10.c -o exp10.exe
> exp10.exe
Enter expression: a+b*c
Parsing succeeded: valid expression

> exp10.exe
Enter expression: (a+b)*c
Parsing succeeded: valid expression

> exp10.exe
Enter expression: a+*b
Parsing failed: invalid expression
```

VIVA QUESTIONS

Q1. What is a recursive descent parser?

A: A top-down parsing technique that uses a set of recursive procedures, one per non-terminal, to process the input and build a parse tree.

Q2. What is the role of next()?

A: It reads the next token from the input and stores it in the lookahead variable (token) so that the parsing functions can test it.

Q3. What are the main problems of recursive descent parsers?

A: They cannot handle left-recursive grammars, may need backtracking if the grammar is not left-factored, and error recovery is difficult.

EXPERIMENT 11

Recursive descent parser for S → (L) | a, L → L,S | S

AIM

Write a recursive descent parser for the grammar given below.

```
S → ( L ) | a
L → L , S | S
```

Note: After removing left recursion: L → S L', L' → , S L' | ε. The loop in L() implements L'.

PROGRAM

exp11.c

```c
#include <stdio.h>
#include <ctype.h>

enum {
    TOKEN_A,
    TOKEN_LPAREN,
    TOKEN_RPAREN,
    TOKEN_COMMA,
    TOKEN_END,
    TOKEN_INVALID
} token;

const char *p;

void next()
{
    while (isspace((unsigned char)*p))
        p++;

    if (*p == 'a')
        token = TOKEN_A;
    else if (*p == '(')
        token = TOKEN_LPAREN;
    else if (*p == ')')
        token = TOKEN_RPAREN;
    else if (*p == ',')
        token = TOKEN_COMMA;
    else if (*p == '\0' || *p == '\n')
        token = TOKEN_END;
    else
        token = TOKEN_INVALID;

    if (token != TOKEN_END)
        p++;
}

int S(void);
int L(void);

int S(void)
{
    if (token == TOKEN_A)
    {
        next();
        return 1;
    }
```

```
    if (token == TOKEN_LPAREN)
    {
        next();

        if (L() && token == TOKEN_RPAREN)
        {
            next();
            return 1;
        }
    }

    return 0;
}

int L(void)
{
    if (!S())
        return 0;

    while (token == TOKEN_COMMA)
    {
        next();

        if (!S())
            return 0;
    }

    return 1;
}

int main(void)
{
    char line[256];

    printf("Enter expression: ");

    if (fgets(line, sizeof(line), stdin) == NULL)
        return 1;

    p = line;
    next();

    if (S() && token == TOKEN_END)
        printf("Parsing succeeded: valid expression\n");
    else
        printf("Parsing failed: invalid expression\n");

    return 0;
}
```

COMPILE & RUN

```
> gcc exp11.c -o exp11.exe
> exp11.exe
Enter expression: (a,a)
Parsing succeeded: valid expression

> exp11.exe
Enter expression: (a,(a,a))
Parsing succeeded: valid expression

> exp11.exe
Enter expression: (a,)
Parsing failed: invalid expression
```

VIVA QUESTIONS

Q1. What are the advantages of a recursive descent parser?

A: Easy to understand and write; maps directly to the grammar structure; efficient when the grammar is LL(1).

Q2. What is the difference between a recursive descent parser and a predictive parser?

A: A predictive parser is a recursive descent parser that needs no backtracking: it looks ahead one token to choose the correct production. A basic recursive descent parser may backtrack when a non-terminal has several alternatives.

Q3. What is the role of the lookahead token?

A: It holds the current input symbol; the parser compares it with the expected terminal, consumes it and moves to the next symbol, or reports a syntax error.

EXPERIMENT 12

C program to calculate the FIRST function of a grammar

AIM

Write a C program to calculate the FIRST function for the grammar given below.

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

Note: After removing left recursion: E → T E', E' → + T E' | ε, T → F T', T' → * F T' | ε, F → (E) | id. Ep and Tp in the program stand for E' and T'.

PROGRAM

exp12.c

```c
#include <stdio.h>
#include <string.h>

char E[10], Ep[10], T[10], Tp[10], F[10];

void add(char *s, char c)
{
    int i = strlen(s);
    s[i] = c;
    s[i + 1] = '\0';
}

void compute()
{
    E[0] = Ep[0] = T[0] = Tp[0] = F[0] = '\0';

    add(F, '(');
    add(F, 'i');

    strcpy(T, F);

    add(Tp, '*');
    add(Tp, 'e');

    strcpy(E, T);

    add(Ep, '+');
    add(Ep, 'e');
}

void print(char *n, char *s)
{
    int i;

    printf("FIRST(%s) = { ", n);

    for (i = 0; s[i]; i++)
    {
        if (s[i] == 'i')
            printf("id");
        else if (s[i] == 'e')
            printf("epsilon");
        else
```

```
            printf("%c", s[i]);

        if (s[i + 1])
            printf(", ");
    }

    printf(" }\n");
}

int main()
{
    compute();

    print("E", E);
    print("Ep", Ep);
    print("T", T);
    print("Tp", Tp);
    print("F", F);

    return 0;
}
```

COMPILE & RUN

```
> gcc exp12.c -o exp12.exe
> exp12.exe
FIRST(E) = { (, id }
FIRST(Ep) = { +, epsilon }
FIRST(T) = { (, id }
FIRST(Tp) = { *, epsilon }
FIRST(F) = { (, id }
```

VIVA QUESTIONS

Q1. What is FIRST(X)?

A: The set of terminals that can appear as the first symbol of any string derived from X. If X can derive ε, then ε is also in FIRST(X).

Q2. How is ε represented in the program?

A: The character 'e' is stored in the set and printed as epsilon.

Q3. Why are FIRST(E), FIRST(T) and FIRST(F) equal?

A: E → T E' and T → F T' always start with whatever F starts with, and F starts with (or id. So all three are { (, id }.

EXPERIMENT 13

YACC program to implement a top-down parser

AIM

Write a YACC program to implement a top-down parser for the given grammar.

```
S → a A
A → b | ε
```

Note: YACC generates a bottom-up (LALR) parser by default. This program uses a left-factored, LL(1)-style grammar so that the parse can be followed top-down. The scanner yylex() is written by hand.

PROGRAM

exp13.y

```
%{
#include <stdio.h>
%}

%token A B

%%
S : A X { printf("Parser S -> aA\n"); }
  ;

X : B { printf("Parser A -> b\n"); }
  |   { printf("Parser A -> epsilon\n"); }
  ;
%%

int main()
{
    printf("Enter input (a or ab): ");
    yyparse();
    return 0;
}

int yyerror(char *msg)
{
    printf("Syntax error: %s\n", msg);
    return 0;
}

int yylex()
{
    int c = getchar();

    if (c == 'a')
        return A;
    if (c == 'b')
        return B;
    if (c == '\n' || c == EOF)
        return 0;

    return c;
}
```

COMPILE & RUN

```
> bison -d exp13.y
> gcc exp13.tab.c -o exp13.exe
> exp13.exe
Enter input (a or ab): ab
Parser A -> b
Parser S -> aA

> exp13.exe
Enter input (a or ab): a
Parser A -> epsilon
Parser S -> aA
```

VIVA QUESTIONS

Q1. What is YACC?

A: Yet Another Compiler Compiler: a tool that generates a parser from a context-free grammar.

Q2. What is top-down parsing?

A: Parsing that starts from the start symbol and tries to derive the input string.

Q3. Is the parser generated by YACC top-down or bottom-up?

A: Bottom-up (LALR(1)).

EXPERIMENT 14

YACC program to evaluate an algebraic expression

AIM

Write a YACC program to evaluate an algebraic expression.

PROGRAM

algebra.l

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+  { yylval = atoi(yytext); return NUMBER; }
[ \t\n] ;
.       { return yytext[0]; }
%%

int yywrap()
{
    return 1;
}
```

algebra.y

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

%token NUMBER
%left '+' '-'
%left '*' '/'

%%
stmt : expr {
    printf("Evaluated result = %d\n", $1);
}
;

expr : expr '+' expr { $$ = $1 + $3; }
     | expr '-' expr { $$ = $1 - $3; }
     | expr '*' expr { $$ = $1 * $3; }
     | expr '/' expr {
         if ($3 == 0) {
             printf("Divide by zero!\n");
             exit(1);
         }
         $$ = $1 / $3;
     }
     | '(' expr ')' { $$ = $2; }
     | NUMBER { $$ = $1; }
;
%%

int main()
{
    printf("Enter algebraic expression:\n");
    yyparse();
    return 0;
```

```
}

int yyerror(char *msg)
{
    printf("Syntax error: %s\n", msg);
    return 0;
}
```

COMPILE & RUN

```
> flex algebra.l
> bison -dy algebra.y
> gcc lex.yy.c y.tab.c -o algebra.exe
> algebra.exe
Enter algebraic expression:
5 + 3 * (2 - 1)
^Z
Evaluated result = 8
```

VIVA QUESTIONS

Q1. What is YACC?

A: A tool used to generate a parser from a grammar specification.

Q2. What is the purpose of this program?

A: To evaluate algebraic expressions using parsing: each grammar rule carries an action that computes the value ($$ = $1 + $3 etc.).

Q3. What is %token NUMBER?

A: It declares NUMBER as a token that is returned by the Lex scanner.